Reservoir Computing

A modern approach to dynamical system forecast

Alvin B. Lin*, Super Chang*

National Yang Ming Chiao Tung University

Table of Contents

"The Problem"

History of Chaotic System Forecast

Methodologies

- Reservoir Computing

The Future

Conclusion

[1] *Equal contribution

“The Problem”

The Physical Problem: The Butterfly Effect

The Core Concept: **Dynamical System**

The Physical Problem: The Butterfly Effect

The Core Concept: **Dynamical System**

- Three Body System

The Physical Problem: The Butterfly Effect

The Core Concept: **Dynamical System**

- Three Body System
- A tiny disturbance can vary dynamics vitally.

The Physical Problem: The Butterfly Effect

The Core Concept: **Dynamical System**

- Three Body System
- A tiny disturbance can vary dynamics vitally.

The Physical Problem: The Butterfly Effect

The Core Concept: **Dynamical System**

- Three Body System
- A tiny disturbance can vary dynamics vitally.

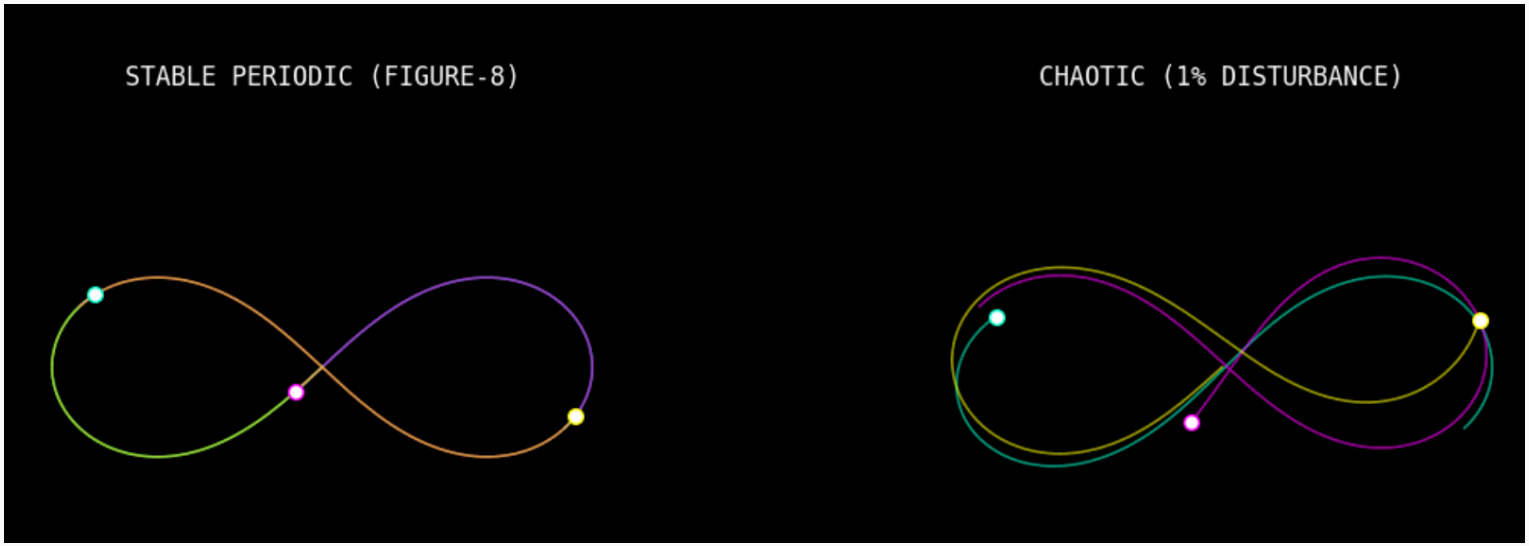

Figure 1: Comparison of 3-body systems' dynamics. *Left:* Periodic Solution (Figure-8). *Right:* Adding 1% disturbance to the initial condition of Figure-8.

So... What Exactly Is a Dynamical System?

Definition

A dynamical system is a triple $(T, \mathcal{M}, \Phi)$:

- T (Time): A monoid (usually $\mathbb{R}$ or $\mathbb{Z}$).
- $\mathcal{M}$ (Phase Space): A manifold or topological space.
- Φ (Evolution Function): $\Phi : U \subseteq T \times \mathcal{M} \to \mathcal{M}$ such that:
  1. $\Phi(0, x) = x$
  2. $\Phi(t, \Phi(s, x)) = \Phi(t + s, x)$

So... What Exactly Is a Dynamical System?

Definition

A dynamical system is a triple $(T, \mathcal{M}, \Phi)$:

- T (Time): A monoid (usually $\mathbb{R}$ or $\mathbb{Z}$).
- $\mathcal{M}$ (Phase Space): A manifold or topological space.
- Φ (Evolution Function): $\Phi : U \subseteq T \times \mathcal{M} \to \mathcal{M}$ such that:
  1. $\Phi(0, x) = x$
  2. $\Phi(t, \Phi(s, x)) = \Phi(t + s, x)$

Find it hard to imagine?

So... What Exactly Is a Dynamical System?

Definition

A dynamical system is a triple $(T, \mathcal{M}, \Phi)$:

- T (Time): A monoid (usually $\mathbb{R}$ or $\mathbb{Z}$).
- $\mathcal{M}$ (Phase Space): A manifold or topological space.
- Φ (Evolution Function): $\Phi : U \subseteq T \times \mathcal{M} \to \mathcal{M}$ such that:
  1. $\Phi(0, x) = x$
  2. $\Phi(t, \Phi(s, x)) = \Phi(t + s, x)$

Find it hard to imagine?

Example

1. Autonomous ODE:

$$\dot{\mathbf{x}}(t) = f(\mathbf{x}(t)), \quad \mathbf{x}(0) = \mathbf{x}_0$$

   is a continuous **dynamical system**.

2. Recurrence series: $a_{n+1} = g(a_n)$ is a discrete **dynamical system**.

Chaotic Behaviour

Definition

If a dynamical system $(T, \mathcal{M}, \Phi)$ satisfies the following conditions on a compact invariant set $V \subset \mathcal{M}$, it is said to be chaotic.

Chaotic Behaviour

Definition

If a dynamical system $(T, \mathcal{M}, \Phi)$ satisfies the following conditions on a compact invariant set $V \subset \mathcal{M}$, it is said to be chaotic.

1. $\exists\, \delta \in \mathbb{R}^+$ such that $\forall\, x \in V$, $\varepsilon \in \mathbb{R}^+$, $\exists\, y \in B_x(\varepsilon)$, $t \in \mathbb{R}^+$ such that:

$$d(\Phi(t, x), \Phi(t, y)) > \delta$$

Chaotic Behaviour

Definition

If a dynamical system $(T, \mathcal{M}, \Phi)$ satisfies the following conditions on a compact invariant set $V \subset \mathcal{M}$, it is said to be chaotic.

1. $\exists\, \delta \in \mathbb{R}^+$ such that $\forall\, x \in V$, $\varepsilon \in \mathbb{R}^+$, $\exists\, y \in B_x(\varepsilon)$, $t \in \mathbb{R}^+$ such that:
$$d(\Phi(t, x), \Phi(t, y)) > \delta$$
2. For any two open sets $U, W \subset V$, $\exists\, t > 0$ such that:
$$\Phi(t, U) \cap W \neq \emptyset$$
3. The set of periodic points $\{x \in V \mid \Phi(t, x) = x \text{ for some } t > 0\}$ is dense in V.

The first condition yields a conclusion: A chaotic system is extremely sensitive to the initial condition, even if there is a tiny turbulence.

Why Do We Need to Keep Track on Chaotic System?

Prediction Does matter, but...

- Focus on Value Forecast: One mistake triggers all moves wrong.

Why Do We Need to Keep Track on Chaotic System?

Prediction Does matter, but...

- Focus on Value Forecast: One mistake triggers all moves wrong.
- Economy: The US or the World?

Why Do We Need to Keep Track on Chaotic System?

Prediction Does matter, but...

- Focus on Value Forecast: One mistake triggers all moves wrong.
- Economy: The US or the World?

Why Do We Need to Keep Track on Chaotic System?

Prediction Does matter, but...

- Focus on Value Forecast: One mistake triggers all moves wrong.
- Economy: The US or the World?

The Goal is not Prediction; it is Reconstruction.

Why Do We Need to Keep Track on Chaotic System?

Prediction Does matter, but...

- Focus on Value Forecast: One mistake triggers all moves wrong.
- Economy: The US or the World?

The Goal is not Prediction; it is Reconstruction.

- Weather: Hurricane # vs. Climate

Problem Statement

Conditions: Consider a continuous-time dynamical system defined on a d-dimensional compact manifold $\mathcal{M} \subseteq \mathbb{R}^d$, where the evolution of the state $\mathbf{x}(t)$ is governed by a smooth flow $\phi : \mathcal{M} \to \mathcal{M}$.

Let $\omega : \mathcal{M} \to \mathbb{R}$ be a smooth scalar observation function and $m \in \mathcal{M}$ be the initial condition such that the observed time series is given by:

$$z_t = \omega(\phi^t(m)), \quad \forall\ t \in \mathbb{N} \cup$$

Given a discrete sequence of observations $\{z_t\}_{t=1}^{T}$ sampled at intervals Δt, where $z_t = z(t \cdot \Delta t)$.

The objective: Find a predictor $\mathcal{F}$ such that for a finite horizon $n > 0$:

$$\hat{z}_{T+n} = \mathcal{F}\big(\{z_t\}_{t=1}^{T} \cup \{\hat{z}_t\}_{t=T+1}^{T+n-1}\big)$$

Subject to the minimisation of the error $|z_{T+n} - \hat{z}_{T+n}|$.

Example of The Problem

Suppose we only have x-component information of an arbitrary $\mathbb{R}^3$ dynamical system, and we don't know how the system looks like. Can we use finite sampled data $\{x_t\}_{t=0}^{T}$ to predict $x_{T+1}, x_{T+2}, \ldots$ well?

Here, we have the following correspondences:

- $d = 3$ and $\mathcal{M} \subseteq \mathbb{R}^3$.
- $\omega(x, y, z) = x$.
- $m \in \mathcal{M}$ given and ϕ unknown.
- $x_s = \omega(\phi^s(m))$

History of Chaotic System Forecast

History of Chaotic System Forecast

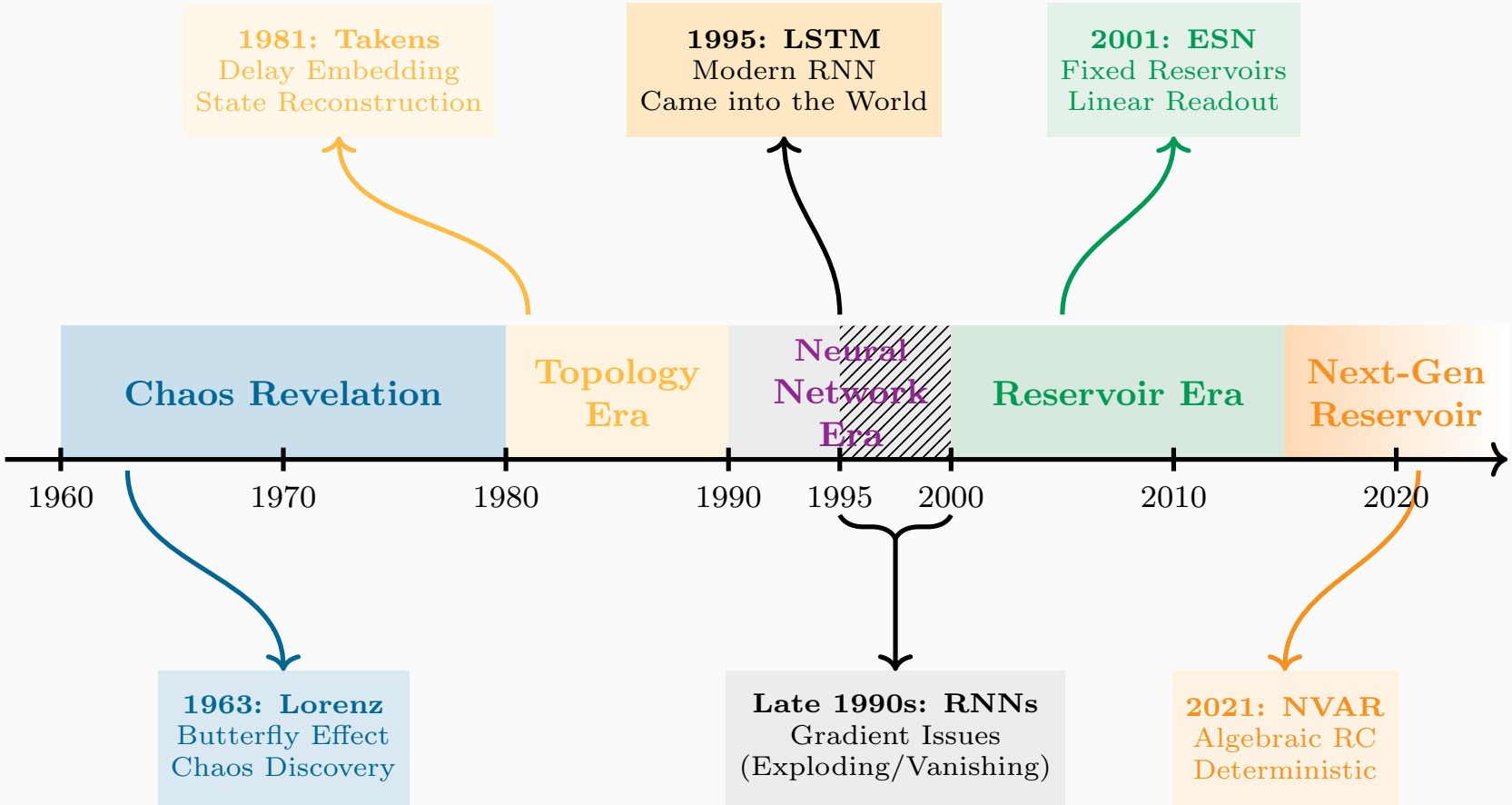

Figure 2: Development of chaotic system forecast.

Lorenz System: A Classic Dynamical System

In 1963, Edward Lorenz developed a simplified mathematical model for atmospheric convection. The system (Lorenz63 system)

$$\begin{cases} \frac{\mathrm{d}u}{\mathrm{d}t} = \sigma(v - u) \\ \frac{\mathrm{d}v}{\mathrm{d}t} = u(\rho - w) - v \\ \frac{\mathrm{d}w}{\mathrm{d}t} = uv - \beta w \end{cases}$$

is chaotic, when parameters are chosen as $\sigma = 10$, $\rho = 28$ and $\beta = \frac{8}{3}$.

The Butterfly

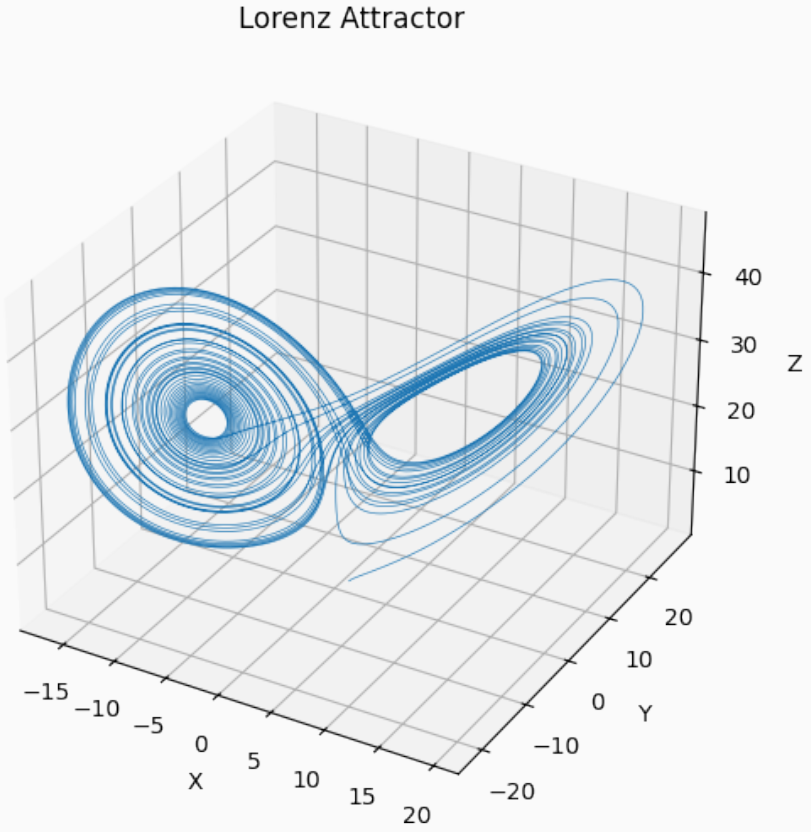

Figure 3: Lorenz Attractor, with symmetric shape that closely resembles a butterfly with two wings.

Methodologies

Methodology: RNN/LSTM

In the late 1990's, people started using RNN or LSTM for the forecast. As in the following recurrence relation:

$$\mathbf{x}_t = f(\mathbf{W}_{\text{in}}\mathbf{x}_{t-1} + \mathbf{B}\mathbf{s}_t + \mathbf{h}),$$

where f: element-wise activation function, $\mathbf{s}_t$: input state, $\mathbf{x}_t$: latent state, our aim is to optimise input matrices $\mathbf{W}_{\text{in}}, \mathbf{B}$ and bias $\mathbf{h}$ through "Backpropagation Through Time" (BPTT).

However, Bengio, Simard, and Frasconi (1994)[1] demonstrated the inherent difficulty of learning long-term dependencies using gradient descent. Therefore we need some innovative ways.

[1] Y.Bengio, P. Simard, and P. Frasconi. "Learning long-term dependencies with gradient descent is difficult". In: IEEE Transactions on Neural Networks 5.2 (1994), pp. 157–166.

Takens' Embedding Theorem

Before moving on, we first introduce a crucial theorem:

Taken's Embedding Theorem

Let $\mathcal{M}$ be a compact manifold of dimension $q \in \mathbb{N}$ and let $\phi \in \mathrm{Diff}(\mathcal{M})$ be a $\mathcal{C}^2$-diffeomorphism and $\omega : \mathcal{M} \to \mathbb{R}$ be a smooth function.

It is a generic property that the $(2q+1)$–delay observation map $\Phi_{(\phi,\omega)} : \mathcal{M} \to \mathcal{M}_x \subseteq \mathbb{R}^{2q+1}$ given by

$$\Phi_{(\phi,\omega)}(x) = (\omega(x), \omega \circ \phi(x), \ldots, \omega \circ \phi^{2q}(x))$$

is an embedding.

This theorem tells us that a delay map, which maps a state to a tuple containing the consecutive states, including the original, forms an embedding.

For ESN and NVAR/NG-RC, we require the foundation of this theorem.

Example of Takens' Theorem

Based on the Takens' Theorem, we take $q = 3$ as its dimension, which means, if we have $N \geq 7$, the map $\Phi : \mathcal{M} \to \mathcal{M}_x \subseteq \mathbb{R}^N$,

$$\Phi_{(\phi,\omega)}(x) = (\omega(x), \omega \circ \phi(x), \ldots, \omega \circ \phi^{N-1}(x))$$

is an embedding, where $\phi(x_t) = \phi(u_t, v_t, w_t) = (u_{t+1}, v_{t+1}, w_{t+1}) = x_{t+1}$. And ω can be chosen as projection functions, e.g. $\omega : \mathbb{R}^3 \to \mathbb{R}$:

$$\omega \begin{pmatrix} u \\ v \\ w \end{pmatrix} = u$$

Therefore, in the context, such Φ can be written in a less complex form:

$$\Phi_{(\phi,\omega)}(x_t) = \mathbf{u}_t := (u_t, u_{t+1}, \ldots, u_{t+N-1}), \qquad \forall\, t \in \mathbb{N} \cup \{0\}.$$

In application, we choose N to be large enough (e.g. $N = 20$) to reach the requirement of the theorem.

Visualisation of Takens' Embedding Theorem

Choose $N = 3$ to see what happens. (Note: Φ is not an embedding.)

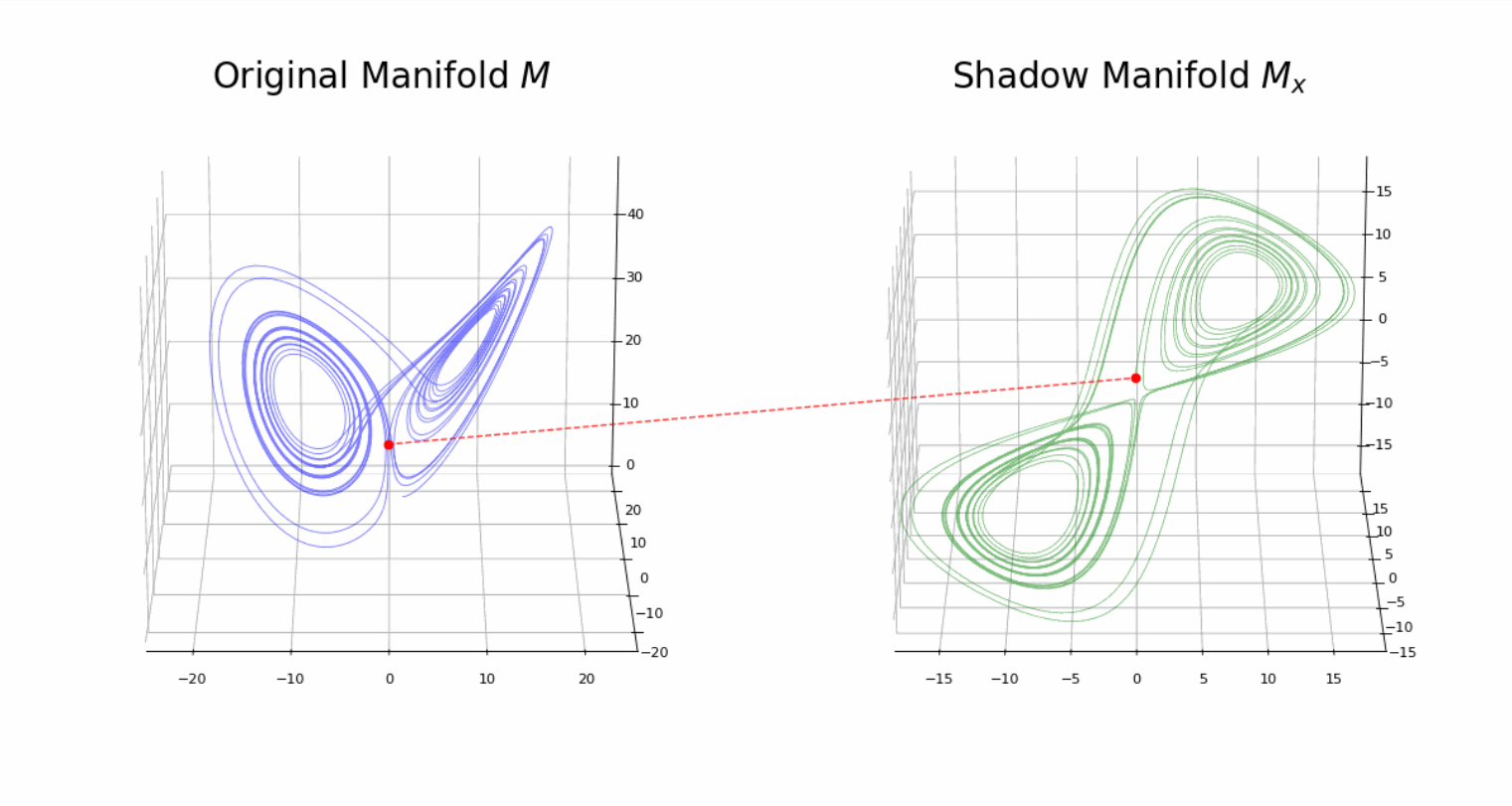

Figure 4: Visuals of Takens' Theorem. *Left:* The original Lorenz manifold $\mathcal{M}$. *Right:* The reconstructed phase space $\mathcal{M}_x$. *Middle:* The red dotted line represents the map Φ.

Reservoir Computing

In this presentation, we are going to pass through two methodologies which are under the field of reservoir computing.

A Reservoir Computer consists of three distinct parts:

- The Input Layer: Maps the external signal (e.g., the u_t from your data) into a high-dimensional space $\mathcal{M}_x$.
- The Reservoir: A large, fixed, and usually random recurrent network. Its weights are never changed during training. It serves as a dynamical memory that "echoes" past inputs.
- The Readout: The only trainable part of the system. It is a simple linear layer that translates the complex state of the reservoir into a prediction.

Echo State Network (ESN)

Based on Takens' Theorem, Grigoryeva et al. (2023)[2] have shown the following network is theoretically able to forecast the dynamical system:

- Determine an appropriate dimension N and training length $T \in \mathbb{Z}^+$.
- Randomly choose $\mathbf{A} \in \mathbb{M}_{N\times N}, \mathbf{C} \in \mathbb{R}^N$.
- Rescale: $\mathbf{A} \leftarrow 0.9 \cdot \frac{\mathbf{A}}{\|\mathbf{A}\|}$, $\mathbf{C} \leftarrow 0.9 \cdot \frac{\mathbf{C}}{\|\mathbf{C}\|}$.
- Recurrence: $\mathbf{x}_t \leftarrow \alpha \tanh(\mathbf{A}\mathbf{x}_{t-1} + \mathbf{C}u_t + b) + (1-\alpha)\mathbf{x}_{t-1}$ for some parameters α, b.
- Gather columns to be matrices: $\mathbf{X} = [\mathbf{x}_1, \ldots, \mathbf{x}_T]$; $\mathbf{Y} = [u_1, \ldots, u_T]$.
- Solve the readout: $\mathbf{W}_{out} = \mathbf{Y}\mathbf{X}^\top(\mathbf{X}\mathbf{X}^\top + \beta\mathbb{I})^{-1}$.

Finally, we do the forecast through $\hat{u}_t = \mathbf{W}_{out}\mathbf{x}_{t-1}$, $\forall\ t = T+1, \ldots$.

[2] Grigoryeva L, Hart A, Ortega JP (2023) Learning strange attractors with reservoir systems. Nonlinearity 36(9):4674.

Structure of ESN

The following figure[3] shows the structure of ESNs:

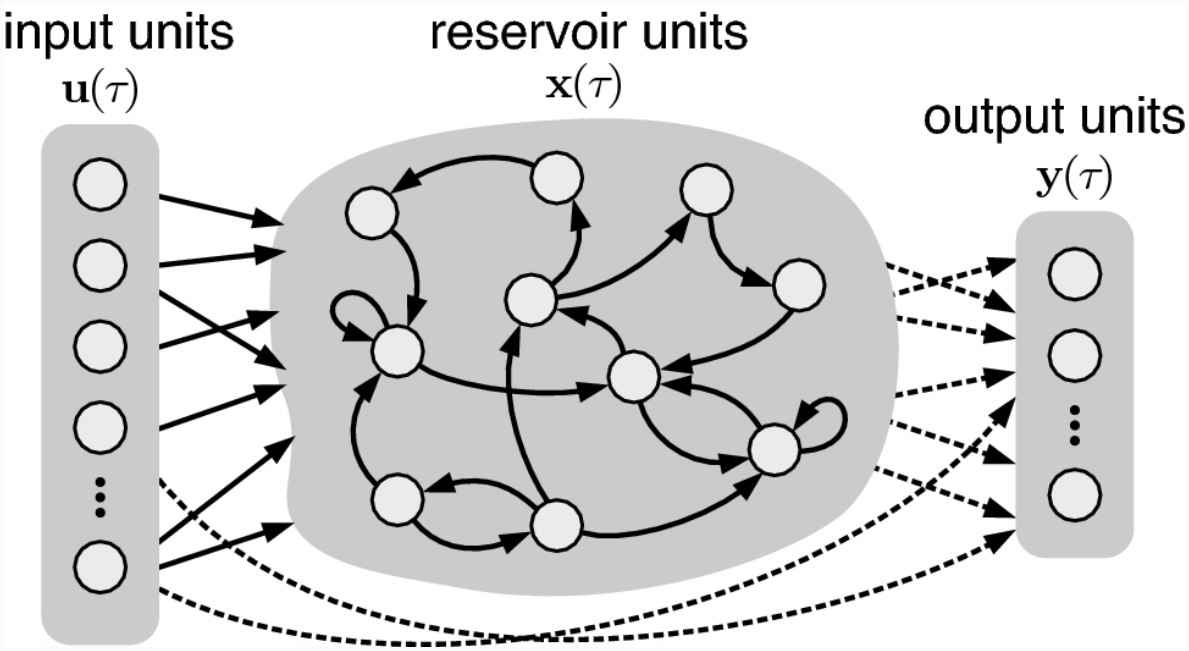

Figure 5: The structure of ESNs.

[3]Hiraga, M., Katada, Y. & Ohkura, K. Echo state networks for embodied evolution in robotic swarms. Artif Life Robotics 28, 139–147 (2023). https://doi.org/10.1007/s10015-022-00828-w

ESN Performance

We tested for some periodic and chaotic systems, the performances are as follows:

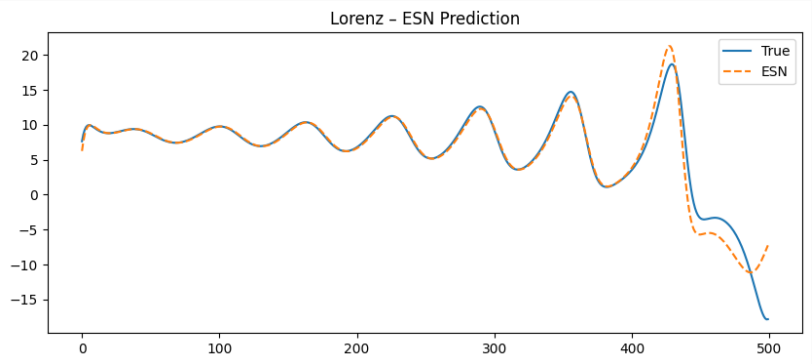

Figure 6: (Chaotic) Prediction on u-component on Lorenz system.

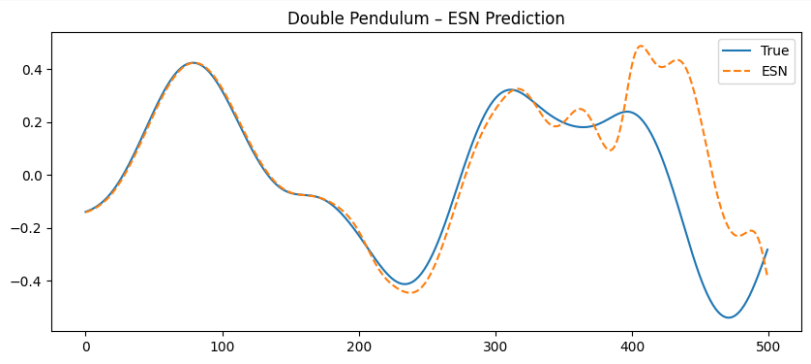

Figure 7: (Chaotic) Prediction on u-component on double pendulum.

From the performance on chaotic systems, we see that the predictions remain accurate for around 200-400 steps, and they capture the chaotic behaviour successfully.

ESN Performance (Cont'd)

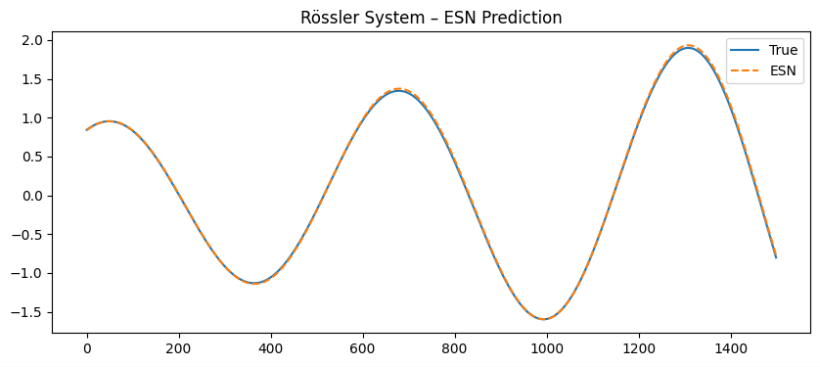

Figure 8: (Chaotic) Prediction on u-component on Rössler System.

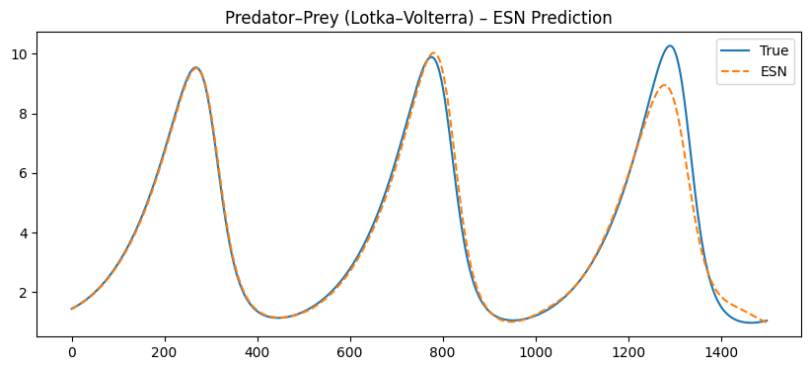

Figure 9: (Periodic) Prediction on u-component on predator prey model.

Next Generation Reservoir Computing (NG-RC)

In recent years, Gauthier et al. (2021)[4] discovered a brand new way of using reservoir computing. This new methodology features a faster execution speed and a higher interpretability. However, the idea is intuitive, take a 3D (normalised) system for example,

$$u(s+\Delta t) \approx u(s) + \Delta t \cdot \frac{\partial u}{\partial t}(s) := u(s) + \Delta t \cdot f(u,v,w).$$

Also, for normalised data, and smoothness on functions f_i, we have:

$$f_i(u,v,w) = \sum_{j,k,l \in \mathbb{Z}^+ \cup \{0\}} a_{ijkl} \cdot u^j \cdot v^k \cdot w^l \approx \sum_{j,k,l \in \mathbb{Z}_d} a_{ijkl} \cdot u^j \cdot v^k \cdot w^l,$$

for some positive integer d. In matrix notation,

$$\begin{pmatrix} u(s+\Delta t) \\ v(s+\Delta t) \\ w(s+\Delta t) \end{pmatrix} = \begin{pmatrix} u(s) \\ v(s) \\ w(s) \end{pmatrix} + \Delta t \cdot \sum_{j,k,l \in \mathbb{Z}_d} \begin{pmatrix} a_{1jkl} \\ a_{2jkl} \\ a_{3jkl} \end{pmatrix} u^j \cdot v^k \cdot w^l$$

[4] Gauthier, D.J., Bollt, E., Griffith, A. et al. Next generation reservoir computing. Nat Commun 12, 5564 (2021). https://doi.org/10.1038/s41467-021-25801-2

NG-RC (Cont'd)

Taylor Expansion reduces the problem to a simple regression, requiring only solving the coefficients, if $\{u^j \cdot v^k \cdot w^l\}_{j,k,l \in \mathbb{Z}_d}$ is known.

NG-RC (Cont'd)

Taylor Expansion reduces the problem to a simple regression, requiring only solving the coefficients, if $\{u^j \cdot v^k \cdot w^l\}_{j,k,l \in \mathbb{Z}_d}$ is known.

Q: What if we have merely u component data?

NG-RC (Cont'd)

Taylor Expansion reduces the problem to a simple regression, requiring only solving the coefficients, if $\{u^j \cdot v^k \cdot w^l\}_{j,k,l \in \mathbb{Z}_d}$ is known.

Q: What if we have merely u component data? Use Takens' Theorem.

NG-RC (Cont'd)

Taylor Expansion reduces the problem to a simple regression, requiring only solving the coefficients, if $\{u^j \cdot v^k \cdot w^l\}_{j,k,l \in \mathbb{Z}_d}$ is known.

Q: What if we have merely u component data? Use Takens' Theorem.

Takens' Thoerem allows us to replace the collection with delays:

$$\mathfrak{S}(t) := \{u(t), \ldots, u^p(t), u(t - \Delta t), \ldots, u(t - k\Delta t), \ldots, u^p(t - k\Delta t)\}$$

for some $p, k \in \mathbb{Z}^+$. We also define $\mathbb{O}(t) := (u(t), \ldots, u^p(t - k\Delta t))^\top$.

NG-RC (Cont'd)

Taylor Expansion reduces the problem to a simple regression, requiring only solving the coefficients, if $\{u^j \cdot v^k \cdot w^l\}_{j,k,l \in \mathbb{Z}_d}$ is known.

Q: What if we have merely u component data? Use Takens' Theorem.

Takens' Thoerem allows us to replace the collection with delays:

$$\mathfrak{S}(t) := \{u(t), \ldots, u^p(t), u(t - \Delta t), \ldots, u(t - k\Delta t), \ldots, u^p(t - k\Delta t)\}$$

for some $p, k \in \mathbb{Z}^+$. We also define $\mathbb{O}(t) := (u(t), \ldots, u^p(t - k\Delta t))^\top$.

Finally, we solve for $b_{i\alpha},\ i \in \{1, 2, 3\},\ \alpha \in I$, such that:

$$Y(s) := u(s + \Delta t) - u(s) = \Delta t \cdot \sum_{\substack{\alpha \in I \\ u_\alpha \in \mathfrak{S}(s)}} b_{1\alpha} u_\alpha = \mathbf{B}^\top \mathbb{O}(s)$$

for a trainable matrix $B = (b_{1\alpha})_{\alpha \in I}$.

NG-RC (Cont'd)

Again, we gather $\{Y(s)\}_{s=0}^{T}$ and $\{\mathbb{O}(s)\}_{s=0}^{T}$ as $\mathbf{Y}$ and $\mathbb{O}$ for regression and solve:

$$\mathbf{B} = \mathbf{Y}\mathbb{O}^{\top}(\mathbb{O}\mathbb{O}^{\top} - \beta\mathbb{I})^{-1},$$

and update as:

$$u(t + \Delta t) \leftarrow u(t) + \mathbf{B}\mathbb{O}(t), \quad \forall\, t \geq T.$$

NG-RC (Cont'd)

Again, we gather $\{Y(s)\}_{s=0}^{T}$ and $\{\mathbb{O}(s)\}_{s=0}^{T}$ as $\mathbf{Y}$ and $\mathbb{O}$ for regression and solve:

$$\mathbf{B} = \mathbf{Y}\mathbb{O}^{\top}(\mathbb{O}\mathbb{O}^{\top} - \beta\mathbb{I})^{-1},$$

and update as:

$$u(t + \Delta t) \leftarrow u(t) + \mathbf{B}\mathbb{O}(t), \quad \forall\, t \geq T.$$

This method does not involve a massive amount of matrix computations compared to traditional ESN.

Concept of NG-RC

The following figure[5] expresses the network structure:

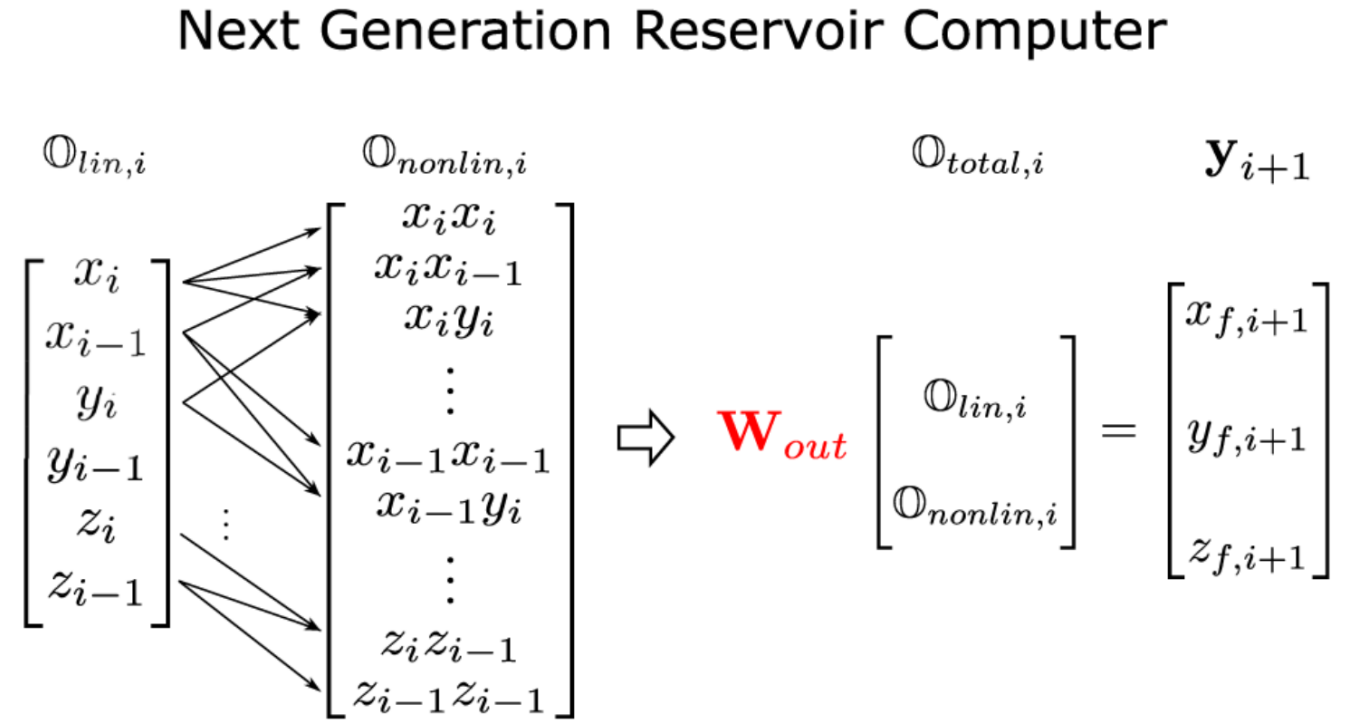

Figure 10: Structure of NG-RC.

[5] Gauthier, D.J., Bollt, E., Griffith, A. et al. Next generation reservoir computing. Nat Commun 12, 5564 (2021). https://doi.org/10.1038/s41467-021-25801-2

NG-RC Performance

NG-RC's performance in both both chaotic and stable systems is precise.

- **Chaotic:**

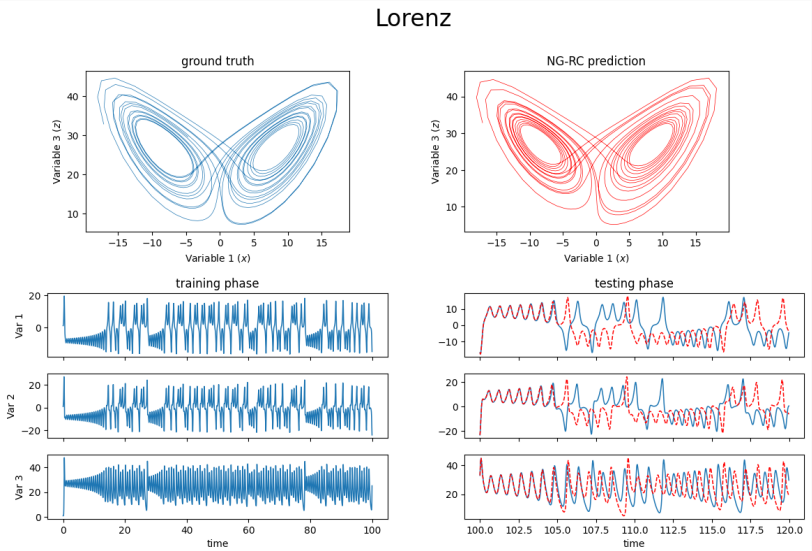

Figure 11: Lorenz system using 3 components. ($\Delta t = 0.025$)

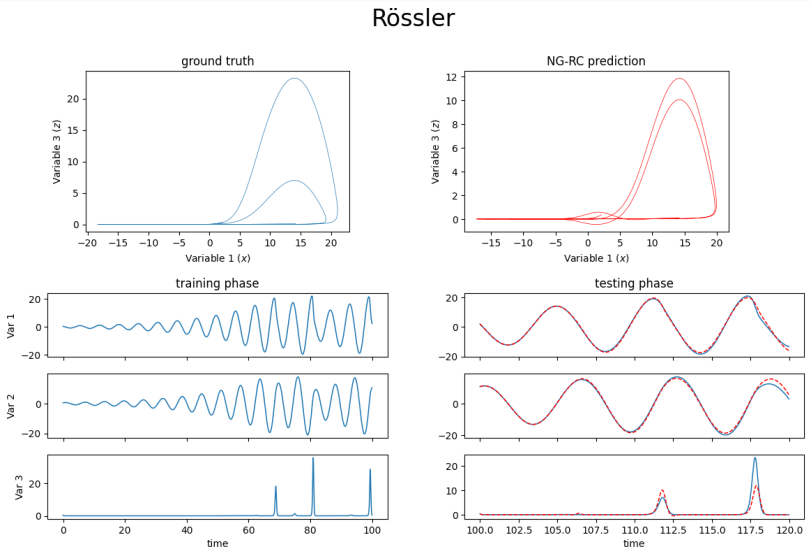

Figure 12: Rössler system with 3 components. ($\Delta t = 0.025$)

NG-RC Performance

- Stable/Periodic:

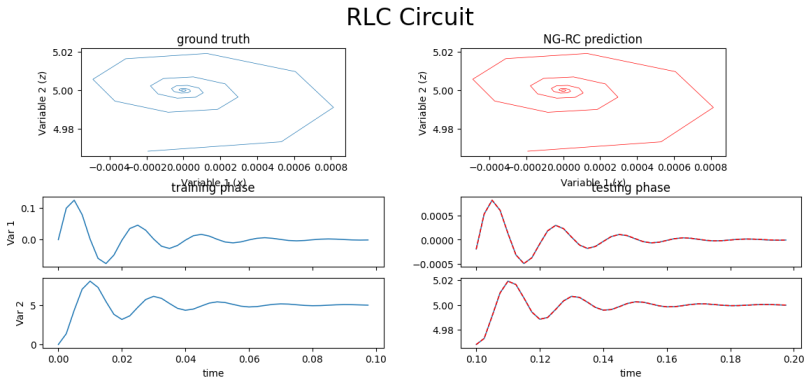

Figure 13: RLC circuit using 2 components. ($\Delta t = 0.025$)

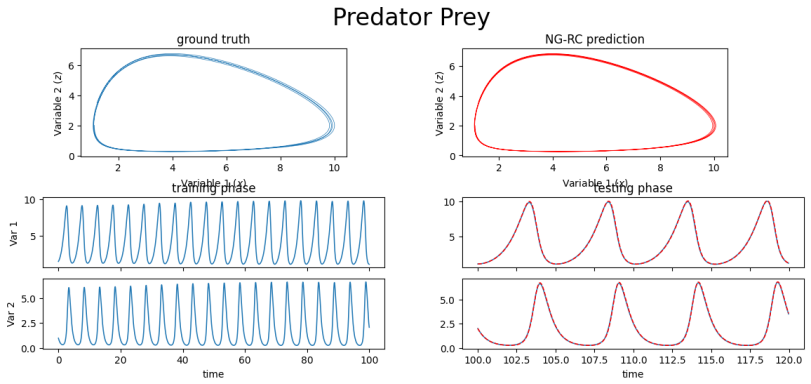

Figure 14: Predator Prey with 2 components. ($\Delta t = 0.025$)

Comparisons between Methodologies

Table 1: Comparison between each methods.

Feature	RNN/LSTM	ESN	NG-RC
Run Time	Slow	Fast	Very fast
Precision	High	High	Very high
Interpretability	Low	Medium	High
Problem	BPTT	Stochastic initialisation	Feature selection (param p, k)

Quick Question: Why Don't We Use Numerical Methods?

Numerical methods (e.g. RK4) can be very helpful and easy to control the errors once we know the equations of system, but:

- They are computationally inconvenient for high dimensional system.

Quick Question: Why Don't We Use Numerical Methods?

Numerical methods (e.g. RK4) can be very helpful and easy to control the errors once we know the equations of system, but:

- They are computationally inconvenient for high dimensional system.
- We generally don't know how do the systems look like.

Quick Question: Why Don't We Use Numerical Methods?

Numerical methods (e.g. RK4) can be very helpful and easy to control the errors once we know the equations of system, but:

- They are computationally inconvenient for high dimensional system.
- We generally don't know how do the systems look like.
- ECG, Electrocardiography.

The Future

Future Work

In the future, here are some future works that could possibly to work on:

- **Physics-Informed NG-RC:** Can we combine NG-RC with "partial" physics? If we know 50% of the equations (e.g., a circuit layout), can we use data to fill in the unmodeled friction or noise?

Future Work

In the future, here are some future works that could possibly to work on:

- **Physics-Informed NG-RC:** Can we combine NG-RC with "partial" physics? If we know 50% of the equations (e.g., a circuit layout), can we use data to fill in the unmodeled friction or noise?
- **Hyperparameter Selection Mechanism**: Closing the gap on Generalized Synchronisation—proving exactly how many delays we need for any system, moving from empirical "tuning" to a universal law of machine memory. Alternatively, we may use **Bayesian Optimisation** to determine the hyperparameters directly.

Conclusion

Summary

1. **The Core Challenge: Why Chaos?**

Summary

1. **The Core Challenge: Why Chaos?**
   - Sensitive Dependence

Summary

1. **The Core Challenge: Why Chaos?**
   - Sensitive Dependence
   - Gradient Conflict

Summary

1. **The Core Challenge: Why Chaos?**
   - Sensitive Dependence
   - Gradient Conflict
2. **Theoretical Breakthrough: Manifold Reconstruction**

Summary

1. **The Core Challenge: Why Chaos?**
   - Sensitive Dependence
   - Gradient Conflict
2. **Theoretical Breakthrough: Manifold Reconstruction**
   - Takens' Embedding Theorem

Summary

1. **The Core Challenge: Why Chaos?**
   - Sensitive Dependence
   - Gradient Conflict
2. **Theoretical Breakthrough: Manifold Reconstruction**
   - Takens' Embedding Theorem
   - Generalized Synchronization

Summary

1. **The Core Challenge: Why Chaos?**
   - Sensitive Dependence
   - Gradient Conflict
2. **Theoretical Breakthrough: Manifold Reconstruction**
   - Takens' Embedding Theorem
   - Generalized Synchronization
3. **Final Conclusion**

Summary

1. **The Core Challenge: Why Chaos?**
   - Sensitive Dependence
   - Gradient Conflict
2. **Theoretical Breakthrough: Manifold Reconstruction**
   - Takens' Embedding Theorem
   - Generalized Synchronization
3. **Final Conclusion**
   - Topology Over Training

Summary

1. **The Core Challenge: Why Chaos?**
   - Sensitive Dependence
   - Gradient Conflict
2. **Theoretical Breakthrough: Manifold Reconstruction**
   - Takens' Embedding Theorem
   - Generalized Synchronization
3. **Final Conclusion**
   - Topology Over Training
   - Efficiency

Summary

1. **The Core Challenge: Why Chaos?**
   - Sensitive Dependence
   - Gradient Conflict
2. **Theoretical Breakthrough: Manifold Reconstruction**
   - Takens' Embedding Theorem
   - Generalized Synchronization
3. **Final Conclusion**
   - Topology Over Training
   - Efficiency
   - Maturity

Questions?

Acknowledgement

Acknowledgement

We are deeply indebted to Professor Lin for his expert instruction and for the stimulating discussions that helped shape the core ideas of this presentation.

More Information

To access the code or more detail content, please go to our *github* repositories through QR codes.

Alvin

Super